

Final

1.graph

存储

- 邻接表
- 相邻矩阵

算法

最短路径

- Dijkstra: 新加入一个就更新一次, 最开始没连的都是inf
- Floyd

最小生成树

- Prim: 取已相连部分之外最小的
- Kruskal算法: 以权的大小为顺序处理各条边, 如果边连接两个不同等价类, 则这条边被添加到MST

证明

对于一个带权连通图, 如果边权值互不相同, 则最小生成树唯一。同学提出的证明:

假设存在两个不同的最小生成树T1和T2。则T1的边集与T2的边集不同: 存在在T1中但不在T2中的边, 设这些边中边权最小的是e1; 存在在T2中但不在T1中的边, 设这些边中边权最小的是e2。因为边权互不相同, 所以 $w(e1) \neq w(e2)$

- 若 $w(e1) < w(e2)$, 将e1加入到T2中形成环, 这个环上, 必然存在一条边f2, f2在T2中但不在T1中。所以 $w(f2) \geq w(e2) > w(e1)$, T2加入e1删除f2得到权重更小的树, 矛盾。
- 若 $w(e1) > w(e2)$, 完全对称证明: 将e2加入到T1中形成环, 这个环上, 必然存在一条边f1, f1在T1中但不在T2中。所以 $w(f1) \geq w(e1) > w(e2)$, T1加入e2删除f1得到权重更小的树, 矛盾

2.内排序

插入排序

- 直接插入排序: 选取一个数字对其**前面**比它大的不断后移, 最终得到自己的位置, 是**稳定的**
- 希尔排序shell: 对相隔某一增量记录组成子表对其进行直接插入排序。每次以增量走遍表进入子表。如果以每次除以2来。最后是对全序列进行直接插入排序。**不稳定**

选择排序

- 直接选择排序: 找最前面的一个, **往后**看是否有更小的, 交换最小的
- 堆排序: 利用堆的性质, 每次取堆顶, 与末尾交换

快速排序

快速排序并不适用于有序数据

- 两个下标分别从序列左端、右端向序列中间扫描，遇见小于/大于的值将逆置记录移动到空闲为止。先左后右，空闲位置就是不断变化的pivot值本身

归并排序

- 空间复杂度最大
- 先分，然后一个个有序合并

基数排序

不基于比较和移动进行排序，而基于关键词各位的大小排序

高位优先和地位优先，分别从低位开始排序或者从高位开始。每一趟分一位一个桶，最后叫收集

10.检索

- 折半查找查找成功平均查找长度为 $O(\log_2(n+1))$

线性探查法：从散列值对应的位置开始向后遍历，插入到第一个空位置 双散列法：使用一个额外的散列函数，发生冲突时对当前位置和这个散列值进行运算得到下一个位置，直到找到空位置

外排序

多路归并

构造huffman(多叉分节点为 $(n-1) \bmod k$),将每一层加上(最上面的只需要加一次所以，加到最底下的)

12.高级数据结构

广义表

- 表示方式，子表第一次出现的时候有元素，最上层的是根节点子树。不标表的话直接括号
- 深度是展开表后所含**括号的**层数。循环表的深度为无穷大，线性表深度为1
- tail返回广义表中去掉第一个元素后剩余的部分。类似lisp递归定义为空表和头部尾部。
- head对表取元素